

NTP 2026 - SpaceSec Shield

Final Technical Report

Phase 3 Security Layer: Lightweight Secure Communication for CubeSat Links

Prepared for: SpaceSec Shield Project Evaluation

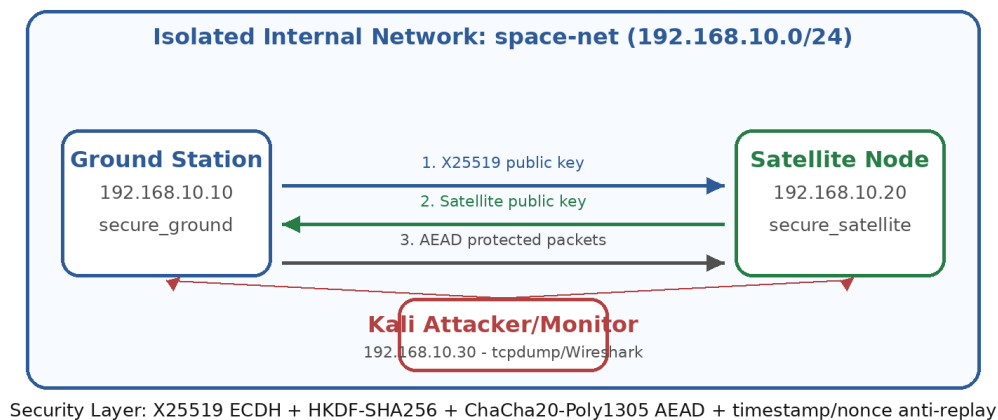
Prepared by: SpaceSec Shield Team

Date: May 2026

Technology Stack

X25519 ECDH - HKDF-SHA256 - ChaCha20-Poly1305 AEAD - UDP - OpenSSL -
VirtualBox - Wireshark/tcpdump

SpaceSec Shield Phase 3 Security Architecture



Cover Figure: validated three-VM SpaceSec Shield Phase 3 architecture.

Executive Summary

SpaceSec Shield is a software-defined security layer designed to protect CubeSat-style ground-to-space communication links under resource constraints. This report documents the Phase 3 implementation and validation of the Security Layer inside the official three-VM environment. The system establishes a fresh session key using ephemeral X25519 Elliptic Curve Diffie-Hellman (ECDH), derives a 32-byte encryption key using HKDF-SHA256, and protects commands using ChaCha20-Poly1305 Authenticated Encryption with Associated Data (AEAD).

The deployed prototype was tested using a Ground Station virtual machine at 192.168.10.10, a Satellite Node virtual machine at 192.168.10.20, and a Kali monitoring/attacker virtual machine at 192.168.10.30. Practical results show that valid encrypted commands were accepted and executed safely, while replayed packets, tampered packets, and expired packets were rejected. Wireshark/tcpdump evidence confirms that UDP traffic was visible on port 9000 but the command payload was not readable as plaintext.

Report Structure

1. Introduction and problem statement
2. Project objectives and related work
3. Methodology and security design
4. Implementation and simulation environment
5. Attack scenarios, testing, and results
6. Challenges, conclusion, future work, and references

1. Introduction

Miniaturized satellites such as CubeSats reduce the cost and complexity of space missions, but their size, weight, and power constraints limit onboard processing, memory, and energy budgets. NASA describes CubeSats as satellites that must follow specific constraints for shape, size, and weight, enabling lower-cost development and commercial off-the-shelf components (NASA CubeSat Launch Initiative, 2017). These same engineering advantages create cybersecurity challenges when conventional, resource-intensive security controls are applied directly to low-power spacecraft platforms.

Ground-to-satellite communication often relies on radio frequency (RF) links. Without suitable cryptographic protection, an adversary can observe packets, replay previously valid commands, modify packet fields, or attempt unauthorized command injection. Because command links can directly influence spacecraft behavior, even a small authentication weakness can become mission critical.

2. Problem Statement

The core problem addressed by SpaceSec Shield is the lack of a practical, lightweight, and testable security layer for constrained CubeSat communication. The communication core from Phase 2 provided a working two-way UDP command/response channel, but it was intentionally unencrypted. Phase 3 therefore had to protect command payloads while preserving the low-overhead nature of the existing channel.

The specific technical risks addressed in this phase are:

- **Eavesdropping:** an attacker observes RF/UDP traffic and attempts to read command data.
- **Replay attack:** an attacker captures a valid packet and resends it later.
- **Packet tampering:** an attacker modifies ciphertext, header fields, timestamp, nonce, or authentication tag.
- **Command injection:** an unauthorized node attempts to create or forge command packets.
- **Resource pressure:** the security layer must not depend on heavyweight enterprise security stacks.

3. Objectives

The primary objective was to deploy and validate a lightweight software security layer between the Ground Station and the Satellite Node. The design goals are summarized in Table 1.

Objective	Implementation Decision	Expected Security Outcome
Secure key establishment	Ephemeral X25519 ECDH followed by HKDF-SHA256	Per-session 32-byte key and forward secrecy properties
Payload confidentiality	ChaCha20 stream cipher inside ChaCha20-Poly1305 AEAD	Command data is unreadable to passive observers
Integrity and authenticity	Poly1305 authentication tag over header and ciphertext	Tampered or forged packets fail verification
Replay resistance	Timestamp freshness check and per-session nonce cache	Duplicate and expired packets are rejected
Operational validation	Three-VM environment with tcpdump/Wireshark evidence	End-to-end proof under realistic lab conditions

Table 1. Phase 3 implementation objectives and corresponding mechanisms.

4. Related Work

Space communications security is a recognized standards area. The Consultative Committee for Space Data Systems (CCSDS) defines a Space Data Link Security Protocol for space missions that require authentication and/or confidentiality on space links (CCSDS, 2022). SpaceSec Shield is not a full CCSDS implementation; it is a constrained academic prototype that demonstrates the same core security goals at the application/packet level within a virtual CubeSat testbed.

The cryptographic choices are based on standardized and widely implemented primitives. RFC 7748 defines X25519 as a function that can be used in Elliptic Curve Diffie-Hellman key agreement (Langley et al., 2016). RFC 5869 defines HKDF as an HMAC-based key derivation function suitable for deriving protocol keys from input keying material (Krawczyk & Eronen, 2010). RFC 8439 defines ChaCha20 and Poly1305 as a combined AEAD algorithm that provides encryption with authenticated data (Nir & Langley, 2018).

OpenSSL was selected because it provides tested EVP APIs for X25519, HKDF, and ChaCha20-Poly1305. The OpenSSL documentation states that ChaCha20-Poly1305 uses a 256-bit key, a 96-bit IV/nonce, supports Additional Authenticated Data (AAD), and produces a 128-bit authentication tag (OpenSSL Project, 2024). These properties match the locked design decisions of Phase 3.

The design also aligns with lightweight cryptography principles. NIST lightweight cryptography work emphasizes authenticated encryption for constrained devices such as low-power embedded systems and sensors (NIST, 2024). Although this project uses ChaCha20-Poly1305 rather than Ascon, the design rationale is similar: authenticated encryption should provide security while respecting limited compute and power budgets.

5. Methodology

The methodology followed an incremental validation approach. The team first verified the Phase 2 communication path, then integrated the Phase 3 security modules, and finally tested the system under a controlled attacker/monitoring setup. The method emphasized practical evidence: terminal outputs, packet captures, and Wireshark inspection rather than design-only claims.

The Phase 3 workflow was:

7. Import and configure the three virtual machines on the isolated space-net network.
8. Install development dependencies, especially libssl-dev and g++, on the Ground Station and Satellite Node.
9. Transfer the Phase 3 C++ source code from Kali/GitHub to the Ground Station and Satellite Node.
10. Compile the secure Ground Station and secure Satellite Node using OpenSSL libraries.
11. Start the Satellite Node receiver first on UDP port 9000.
12. Run the Ground Station sender to perform the handshake and send protected packets.
13. Capture UDP traffic from Kali using tcpdump and Wireshark.
14. Validate that accepted, replayed, tampered, and expired packets produce the expected security outcomes.

5.1 Validation Logic

The receiver follows a verify-before-execute policy. A command is executed only after session establishment, timestamp validation, duplicate nonce rejection, and AEAD tag verification. This policy is essential because it prevents the Satellite Node from acting on unauthenticated or stale data.

6. System and Security Design

The deployed architecture consists of three virtual machines connected through the isolated internal network space-net. The Ground Station sends commands, the Satellite Node receives and validates them, and Kali monitors traffic as an attacker/analysis node. Figure 1 summarizes the validated architecture.

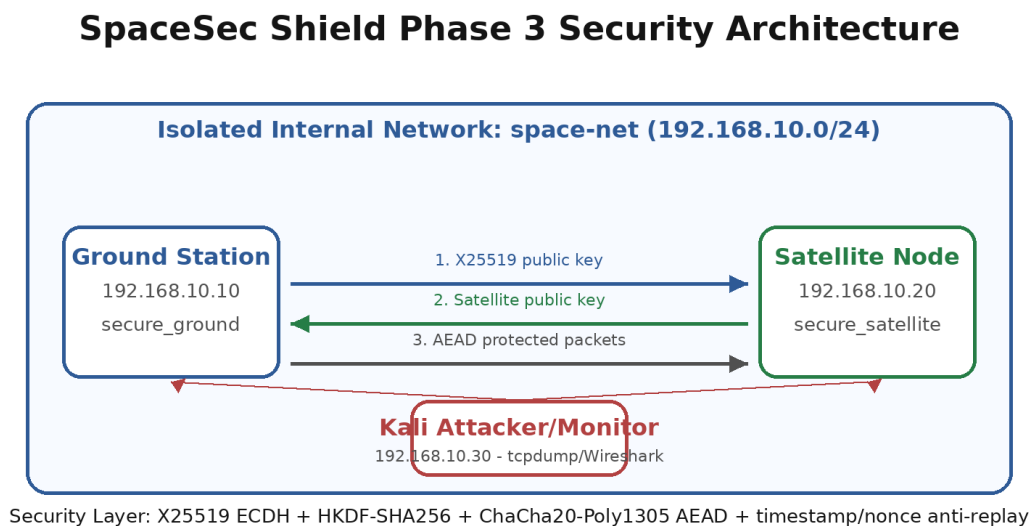


Figure 1. Three-VM SpaceSec Shield Phase 3 architecture and security flow.

6.1 Secure Handshake

The secure handshake uses ephemeral X25519 key pairs. The Ground Station sends its 32-byte public key to the Satellite Node. The Satellite Node sends its 32-byte public key back. Each side combines its private key with the peer public key to derive the same ECDH shared secret. HKDF-SHA256 converts the raw shared secret into a clean 32-byte session key. The session key itself is never transmitted over the network.

6.2 Authenticated Encryption

ChaCha20-Poly1305 is used as an AEAD construction. ChaCha20 encrypts the command payload, while Poly1305 produces a 16-byte authentication tag. The tag covers both the encrypted payload and the AAD header. Therefore, modification of the ciphertext, the session ID, the timestamp, or the nonce causes verification failure.

6.3 Protected Packet Structure

The packet structure is intentionally simple and deterministic. The header fields are visible to support routing and replay validation, but they are authenticated as AAD. The payload is encrypted, and the tag proves integrity and authenticity. Figure 2 shows the implemented packet layout.

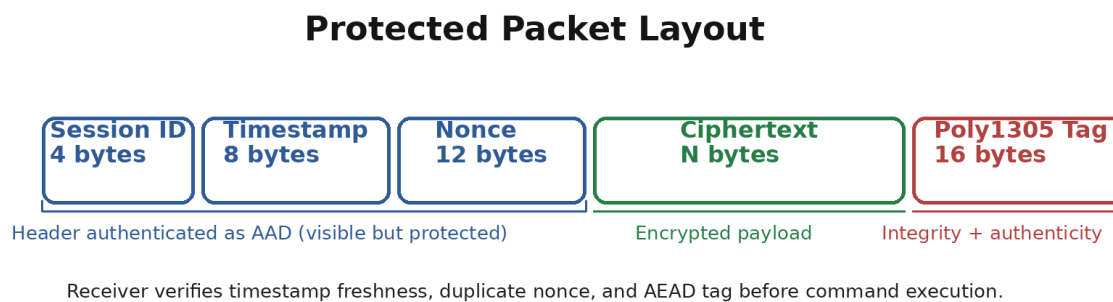


Figure 2. Packet structure: authenticated header, encrypted payload, and Poly1305 tag.

The packet sizes observed during testing match the design: handshake public-key packets were 32 bytes, the valid encrypted command packet was 57 bytes, the tampered packet was 61 bytes, and the expired timestamp packet was 56 bytes. These values are small enough to be suitable for lightweight command channels while still carrying the required security metadata.

6.4 Why AEAD Replaces Standalone SHA

A standalone SHA hash does not authenticate a command because an attacker can modify a message and recompute an unkeyed hash. In contrast, ChaCha20-Poly1305 produces a keyed authentication tag that an attacker cannot forge without the session key. This is why the final design removed standalone SHA for packet integrity and used AEAD instead.

6.5 AES-128-GCM as an Alternative

AES-128-GCM is a valid alternative AEAD mode and is widely standardized. However, it performs best when hardware AES acceleration is available. ChaCha20-Poly1305 is often preferred for software-only, low-power environments because it is efficient without special AES instructions. For a future deployment, AES-128-GCM could be selected where the CubeSat processor provides reliable AES acceleration.

7. Implementation and Simulation Environment

The final validation was performed inside a three-VM lab. The original handover design used an isolated network named space-net. The deployed VirtualBox implementation preserved the same addressing model and role separation. Table 2 summarizes the deployed environment.

VM	Role	IP Address	Evidence/Tool
GroundStation	Mission control / secure UDP sender	192.168.10.10	secure_ground
VMSatelliteNode	Satellite OBC / secure UDP receiver	192.168.10.20	secure_satellite
Kali Linux	Attacker/monitoring node	192.168.10.30	tcpdump and Wireshark

Table 2. Final Phase 3 validation environment.

The Ground Station and Satellite Node were compiled using g++ with the OpenSSL libraries libssl and libcrypto. During setup, libssl-dev was installed on both machines so that OpenSSL header files such as openssl/evp.h were available at compile time. The final commands used were:

```
g++ -std=c++17 secure_ground_station.cpp handshake.cpp crypto.cpp packet.cpp anti_replay.cpp -o secure_ground -lssl -lcrypto
```

```
g++ -std=c++17 secure_satellite_node.cpp handshake.cpp crypto.cpp packet.cpp anti_replay.cpp -o secure_satellite -lssl -lcrypto
```

8. Attack Scenarios

The Phase 3 tests target attack classes that are realistic for unauthenticated RF/UDP command links. Each scenario was executed after the secure handshake to validate the receiver-side controls.

Scenario	Threat Description	Defense Mechanism	Expected Result
Eavesdropping	Attacker captures UDP packets and searches for command plaintext.	ChaCha20-Poly1305 encrypts the payload.	Payload remains unreadable in Wireshark/tcpdump.
Replay attack	Attacker resends an already valid encrypted packet.	Per-session nonce cache detects duplicates.	Duplicate packet is rejected.
Packet tampering	Attacker modifies encrypted payload or authentication tag.	AEAD tag verification fails.	Tampered or forged packet is rejected.
Expired command	Attacker delays and resends a packet outside the freshness window.	Timestamp freshness validation.	Expired timestamp is rejected.
Command injection	Unauthorized node attempts to forge a new	Unknown attacker cannot produce a valid	Forged command is rejected.

	command.	AEAD tag without session key.	
--	----------	-------------------------------	--

Table 3. Security scenarios tested in Phase 3.

9. Testing and Results

The final test run demonstrated the full secure workflow. The Satellite Node accepted the valid authenticated command and rejected replayed, tampered, and expired packets. The output evidence is summarized in Table 4.

Test Case	Input/Action	Observed Result	Status
TC-01 Secure handshake	Ground and Satellite exchange X25519 public keys.	Both sides report secure handshake completed and 32-byte session key established.	PASS
TC-02 Valid command	Ground sends encrypted CMD:CAPTURE_IMAGE packet.	Satellite accepts authenticated command and executes safely.	PASS
TC-03 Replay	Ground resends the same packet.	Satellite rejects duplicate nonce.	PASS
TC-04 Tampered payload	Ground sends modified encrypted payload.	Satellite reports authentication failed: tampered or forged packet.	PASS
TC-05 Expired timestamp	Ground sends packet with expired timestamp.	Satellite rejects expired timestamp.	PASS
TC-06 Packet inspection	Kali captures UDP traffic and searches for plaintext command.	Packets are visible but plaintext command string is not contained in converted data.	PASS

Table 4. Phase 3 security test cases and observed results.

9.1 Ground Station Evidence

Figure 4 shows the Ground Station sending the public key, completing the secure handshake, establishing the 32-byte session key, and sending the four test packets: valid encrypted command, replayed packet, tampered encrypted payload, and expired timestamp packet.

```
spacesec@spacesec-virtual-machine:~/spacesec_phase3$ ./secure_ground
=== SpaceSec Shield Secure Ground Station ===
[SEND] Ground Station public key sent.
[PASS] Secure handshake completed.
[PASS] 32-byte session key established.
[SEND] Valid encrypted command | Size: 57 bytes
[SEND] Replay attack: same packet resent | Size: 57 bytes
[SEND] Tampered encrypted payload | Size: 61 bytes
[SEND] Expired timestamp packet | Size: 56 bytes

=== Ground Station Test Complete ===
spacesec@spacesec-virtual-machine:~/spacesec_phase3$ _
```

Figure 4. Ground Station output: handshake success and protected packet transmission.

The Ground Station packet sequence is important because it proves that the tests were executed from the command-sending side rather than being simulated only inside a single process. The size values printed by the Ground Station also support packet-level analysis of the protected command format.

9.2 Satellite Node Evidence

Figure 5 shows the Satellite Node acting as the security enforcement point. The valid command is accepted and executed, while all malicious or invalid cases are rejected. This confirms the receiver-side verify-before-execute policy.

```
satnode@satnode:~/spacesec_phase3$ ./secure_satellite
=== SpaceSec Shield Secure Satellite Node ===
[INFO] Listening on UDP port 9000...
[INFO] Wireshark filter: udp.port == 9000
[PASS] Secure handshake completed.
[PASS] 32-byte session key established.

[INFO] UDP packet received. Size: 57 bytes
[ACCEPT] Authenticated command received: CMD:CAPTURE_IMAGE
[EXECUTE] Command executed safely.

[INFO] UDP packet received. Size: 57 bytes
[REJECT] Replay detected: duplicate nonce.

[INFO] UDP packet received. Size: 61 bytes
[REJECT] Authentication failed: tampered or forged packet.

[INFO] UDP packet received. Size: 56 bytes
[REJECT] Expired timestamp.

=== Satellite Node Test Complete ===
satnode@satnode:~/spacesec_phase3$
```

Figure 5. Satellite Node output: valid command accepted; replay, tampering, and expired timestamp rejected.

The output demonstrates the exact security decisions required for Phase 3: authenticated commands proceed, while duplicate nonce replay, AEAD authentication failure, and stale timestamp violations are blocked before command execution.

9.3 Packet Analysis and Wireshark Evidence

Kali was used as the external monitoring node. The capture confirms that traffic crossed the isolated network between 192.168.10.10 and 192.168.10.20 on UDP port 9000. Two 32-byte

packets correspond to the public key handshake messages, followed by protected command packets of 57, 57, 61, and 56 bytes.

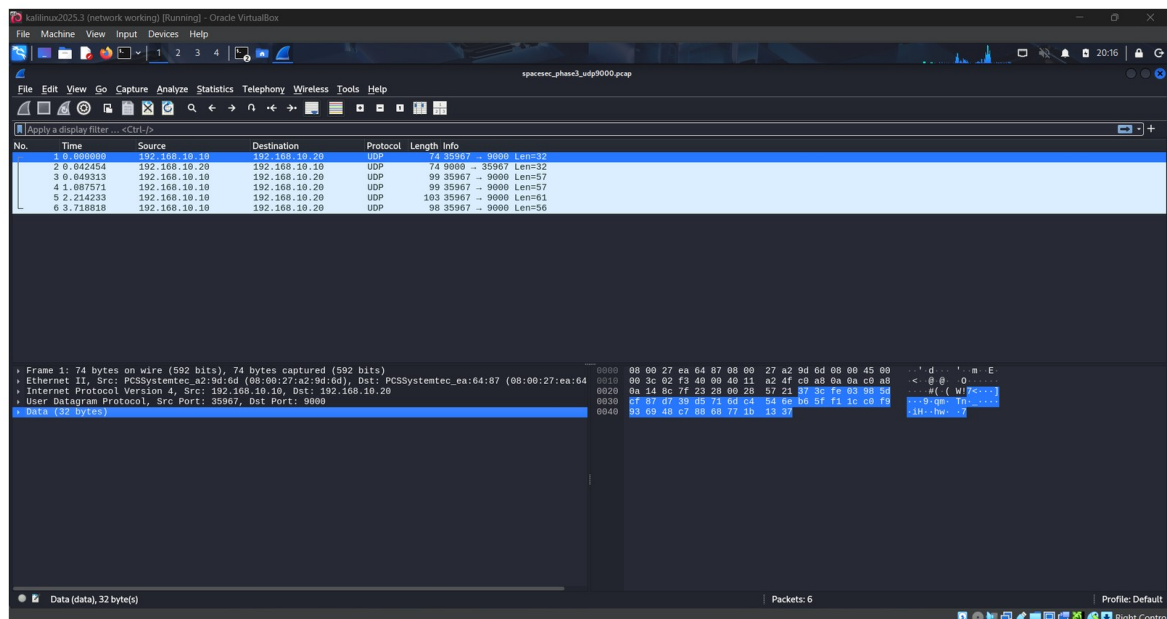


Figure 6. Wireshark packet list: handshake packets and protected UDP command packets on port 9000.

Figure 7 shows the plaintext search test. Wireshark searched the capture for the command string CMD:CAPTURE_IMAGE. The packet bytes were visible, but Wireshark reported that no packet contained that string in converted data. This supports the confidentiality claim: the command was decrypted only at the Satellite Node after successful authentication.

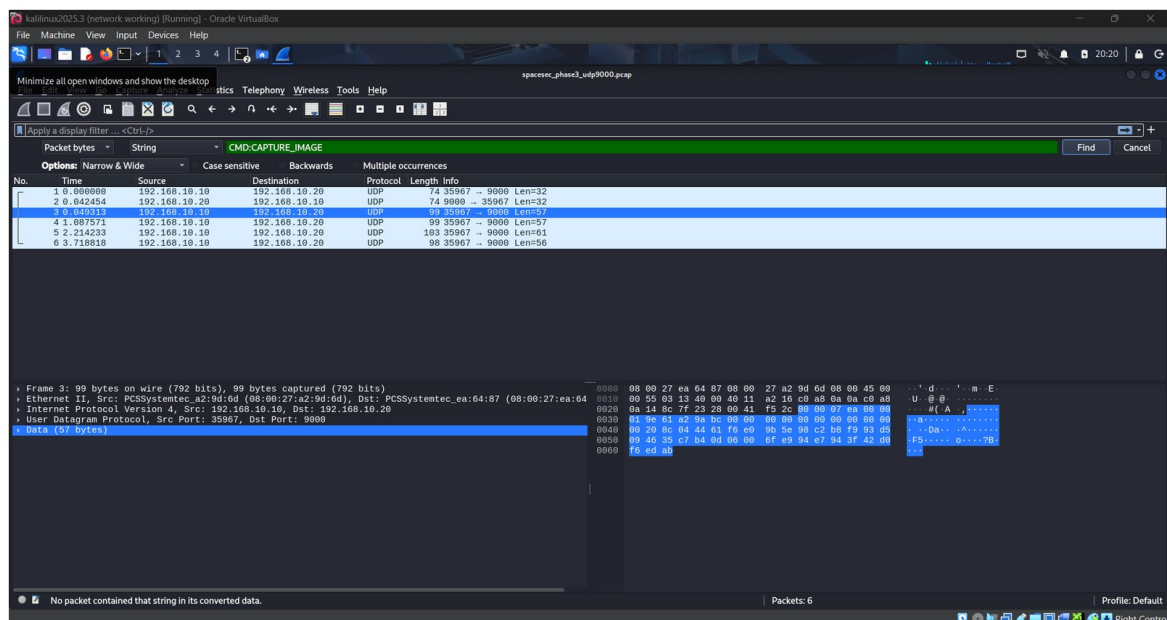


Figure 7. Wireshark evidence: plaintext command search does not appear in captured packet bytes.

9.4 tcpdump Capture Evidence

Kali tcpdump was also used to save a formal packet capture file named `spacesec_phase3_udp9000.pcap`. The captured traffic included six UDP packets: two handshake packets and four test packets. The hex payload did not expose the plaintext command, providing additional packet-level evidence for the Wireshark analysis.

10. Challenges

Several implementation challenges were encountered during the deployment stage:

- VM import differences: the environment was handed over from a VMware-based setup and executed in VirtualBox, requiring careful network recreation.
- Network isolation: the final security test required the internal space-net network, but temporary NAT was needed to install `libssl-dev`.
- Dependency management: OpenSSL runtime was present, but OpenSSL development headers were missing until `libssl-dev` was installed.
- Cross-VM integration: early tests used `127.0.0.1`, which worked locally but failed in the three-VM environment. The Ground Station target had to be changed to `192.168.10.20`.
- Header correctness: `packet.hpp` required explicit `cstdint` inclusion and `std::size_t` to compile consistently across machines.

10.1 Limitations

The implemented handshake provides ephemeral ECDH key agreement and a fresh session key, but it does not fully authenticate the peer identity by itself. In a production system, the exchanged public keys should be authenticated using certificates, pre-shared public keys, or digital signatures to resist man-in-the-middle attacks. Phase 4 can build on this foundation by actively attacking the secured channel.

11. Conclusion

SpaceSec Shield Phase 3 successfully implemented and deployed a lightweight security layer for CubeSat-style communication. The system established a secure session key using X25519 and HKDF-SHA256, protected command packets using ChaCha20-Poly1305 AEAD, authenticated header metadata as AAD, and rejected replayed, tampered, and expired packets. The final validation was not limited to code compilation; it was executed end-to-end inside the official three-VM environment with Wireshark/tcpdump packet evidence.

The results show a practical balance between security and resource constraints. Confidentiality was demonstrated because the command payload did not appear as plaintext in packet captures. Integrity and authenticity were demonstrated because tampered packets were rejected. Replay resistance was demonstrated because duplicate nonces and expired timestamps were rejected. Overall, Phase 3 is ready to support Phase 4 penetration testing and Phase 5 monitoring/dashboard integration.

12. Future Work

- Add authenticated handshake identity using certificates, pre-shared public keys, or digital signatures.
- Extend attack testing in Phase 4 to include spoofing, hijacking attempts, replay automation, and malformed packet fuzzing.
- Add a monitoring dashboard in Phase 5 to visualize accepted/rejected packets, replay counts, and link quality.
- Evaluate performance under controlled propagation delay, jitter, and packet loss using tc/netem.
- Compare ChaCha20-Poly1305 with AES-128-GCM and Ascon-AEAD128 under identical resource conditions.
- Add persistent nonce-cache management and session-expiry policies suitable for longer CubeSat operations.

13. References

- Consultative Committee for Space Data Systems. (2022). Space Data Link Security Protocol (CCSDS 355.0-B-2). <https://ccsds.org/Pubs/355x0b2.pdf>
- Krawczyk, H., & Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF) (RFC 5869). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc5869>
- Langley, A., Hamburg, M., & Turner, S. (2016). Elliptic Curves for Security (RFC 7748). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7748>
- NASA CubeSat Launch Initiative. (2017). CubeSat 101: Basic concepts and processes for first-time CubeSat developers. National Aeronautics and Space Administration. https://www.nasa.gov/wp-content/uploads/2017/03/nasa_csli_cubesat_101_508.pdf
- National Institute of Standards and Technology. (2024). Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions (SP 800-232, initial public draft). <https://csrc.nist.gov/pubs/sp/800/232/ipd>

- Nir, Y., & Langley, A. (2018). ChaCha20 and Poly1305 for IETF Protocols (RFC 8439). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc8439>
- OpenSSL Project. (2024). EVP_chacha20 and ChaCha20-Poly1305 manual page. https://docs.openssl.org/3.3/man3/EVP_chacha20/
- OpenSSL Project. (2024). EVP_CIPHER-CHACHA: ChaCha EVP_CIPHER implementations. https://docs.openssl.org/3.0/man7/EVP_CIPHER-CHACHA/
- SpaceSec Shield Team. (2026). Phase 3 - Security Layer Implementation Task. Internal project specification.